

PARAMETRIC WORLD CARTOGRAPHY EDITOR

Master Product, UX & Technical Design Specification

Canonical foundation document • Revision 1.0 • September 2026

PURPOSE This document is intended to be the authoritative foundation for future design, engineering, prototyping, and AI-assisted development. When a later discussion conflicts with this specification, the conflict should be made explicit and resolved as a design decision rather than silently changing the architecture.

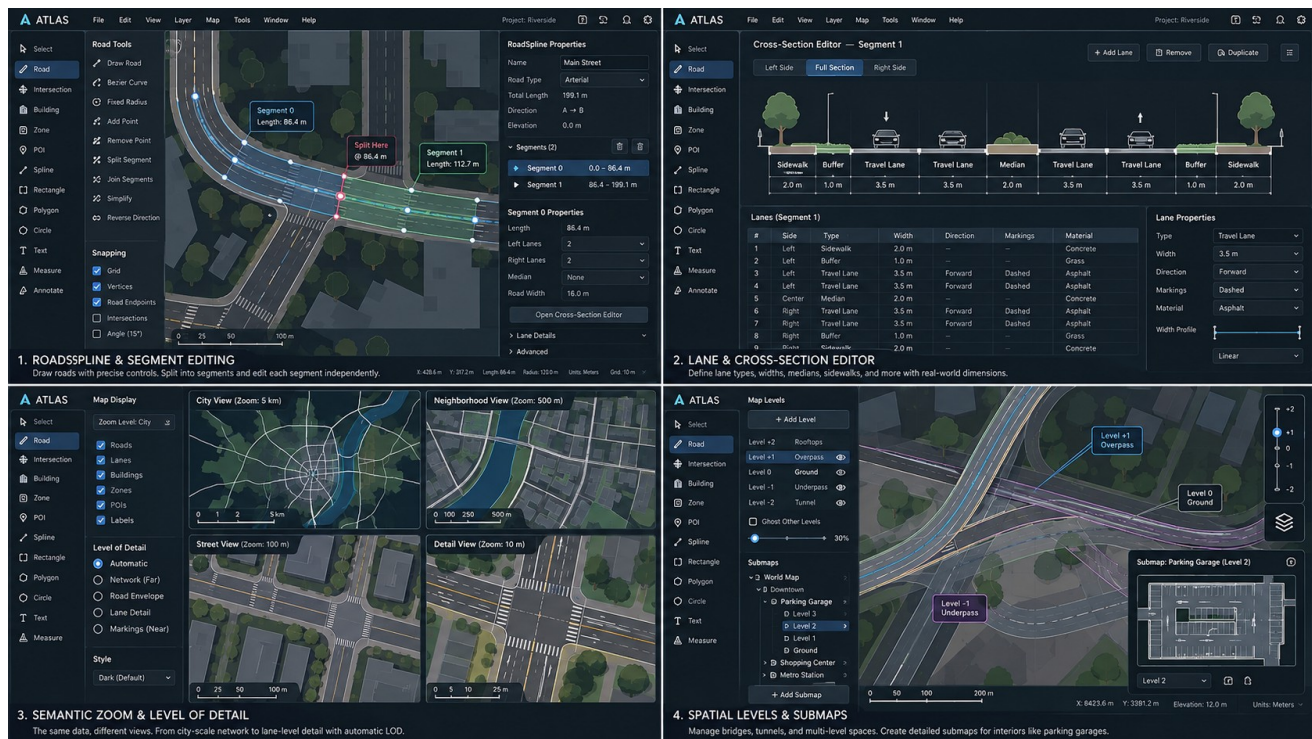


Figure 1. Early UI concept illustrating RoadSpline editing, cross-section editing, semantic zoom, and spatial levels/submaps. Visual styling is illustrative; the data model in this specification is authoritative.

Document Status & Reading Guide

This specification combines a product design document, technical design document, UX specification, and internal wiki. It describes the intended final architecture rather than a promise that every capability belongs in the first implementation.

Label	Meaning
CORE	Architectural concept that should not be casually changed.
MVP	Required to produce a useful first release.
V1	Expected mature first major release capability.
FUTURE	Designed for, but may be implemented later.
DERIVED	Generated from authoritative source data; should normally not be directly edited.

Authoritative design invariants

- Real-world units are authoritative; screen pixels are presentation only.
- Roads are semantic, lane-native structures generated around a directional centerline.
- A newly drawn RoadSpline contains exactly one RoadSegment spanning its full centerline domain.
- RoadSegments are intervals on a parent RoadSpline and may be split or merged without requiring separate centerline splines.
- Lanes and roadside cross-section elements belong to RoadSegments; rendered road polygons are derived geometry.
- Physical geometry and semantic network connectivity are distinct but linked systems.
- Intersections, merges, diverges, transitions, and prefabs operate on semantic connectivity, not only visual overlap.
- Display layers, spatial/elevation levels, and hierarchical submaps are separate concepts.
- The application is game-agnostic. Game-specific meaning is expressed through project-defined types, schemas, metadata, icons, styles, and exporters.
- Editing should remain non-destructive wherever practical; generated objects retain parameters and dependencies.

Source lineage

The road architecture is deliberately inspired by the supplied 2023 senior thesis, which documented an Unreal Engine SBRoadEditor based on a directional Bezier RoadSpline, left/right lane arrays, per-lane width control, generated lane geometry, endpoint intersections, side connections, and repeated spline actors. This specification evolves those ideas into a standalone, game-agnostic 2D/2.5D planning system with explicit RoadSegments, semantic lane connectivity, vertical hierarchy, submaps, semantic zoom, and parametric prefabs. The thesis is design lineage, not a requirement to preserve its Unreal-specific implementation.

1. Product Definition

1.1 Purpose

The application is a purpose-built cartographic and road-network design environment for games, simulations, virtual production, and fictional-world planning. It combines the immediacy of a vector illustration tool, the dimensional rigor of CAD, and the semantic structure of a transportation-network editor.

1.2 What it is

- A 2D-first vector map editor operating in real-world coordinates.
- A lane-native road and transportation-network authoring tool.
- A hierarchical map system supporting elevation, over/under relationships, interiors, and nested submaps.
- A configurable semantic world-design database for POIs, zones, routes, buildings, and project-defined entities.
- A platform for reusable parametric road transitions, junctions, and interchange prefabs.

1.3 What it is not

- Not a 3D level editor or replacement for Unreal/Unity.
- Not a GIS package whose primary purpose is editing real-world geographic databases.
- Not a raster painting application.
- Not a civil-engineering certification tool; real-world units and geometry are supported, but code compliance is project-defined.
- Not game-specific: races, police, quests, factions, and similar concepts are templates or metadata, not hard-coded core types.

1.4 Primary users

User	Primary need
World designer	Lay out cities, roads, districts, interiors, and navigable spaces.
Level designer	Author lane topology, junctions, gameplay routes, and spatial relationships.
Environment artist	Use road/block geometry and reference overlays to guide asset production.
Technical designer	Define semantic connections, metadata, prefabs, and export rules.
Simulation developer	Author realistic road networks with lane-level connectivity.
Game/UI designer	Prototype minimaps, world maps, POIs, zones, and player-context display rules.

2. Conceptual Architecture

2.1 Object hierarchy

```
PROJECT
├─ ProjectDefinitions
├─ Map[]
│   └─ SpatialLevel[]
│       └─ DisplayLayer[]
│           └─ MapObject[]
│               └─ Network objects / relationships
│                   └─ ChildMap[]
└─ Libraries / Styles / Export Profiles
```

2.2 Transportation hierarchy

```
RoadSpline (continuous directional centerline)
├─ RoadSegment [s0,s1]
│   └─ LeftCrossSectionElement[]
│       └─ CenterCrossSectionElement[]
│           └─ RightCrossSectionElement[]
│               └─ Lane / sidewalk / median / shoulder / buffer / custom
└─ RoadSegment ...
```

```
TransportNetwork
├─ ConnectionNode[]
├─ LaneConnection[]
├─ RoadTransition[]
├─ Junction[]
└─ PrefabInstance[]
```

2.3 Three kinds of hierarchy

Hierarchy	Purpose	Example
Display layers	Visual organization and authoring visibility.	Roads, Buildings, Water, Gameplay
Spatial levels	Vertical/navigational context.	Ground, Overpass +1, Tunnel -1
Submaps	Nested spaces with their own geometry/detail.	World → Mall → Parking Garage → B2

CRITICAL DISTINCTION A road can be on the Roads display layer, assigned to Spatial Level +1, and also exist inside a Parking Garage child map. These properties answer different questions and must not be conflated.

3. Editor Shell & Interaction Model

3.1 Main workspace

Region	Role
Top menu	File, Edit, View, Layer, Map, Network, Tools, Window, Help.
Primary toolbar	Selection, Road, Junction, Building, Zone, POI, Spline, Polygon, Measure, Annotate.
Context toolbar	Changes with active tool/selection: snapping, radius, lane preset, segment actions, etc.
Canvas	Primary 2D vector workspace; optional perspective preview is secondary, not authoritative.
Hierarchy panel	Maps, submaps, spatial levels, display layers, groups, objects.
Inspector	Properties of selected object; supports multi-edit and inherited/default values.
Status bar	Coordinates, active level, units, grid, snap state, zoom, current LOD, warnings.
Optional bottom panel	Connectivity graph, cross-section editor, diagnostics, history, or search results.

3.2 Interaction principles

- Direct manipulation first; numeric precision always available.
- Selection determines contextual commands rather than exposing every command at all times.
- Hover previews should show the result of destructive/topological actions before commit.
- Snapping must communicate what is being snapped to: grid, vertex, spline, endpoint socket, lane boundary, intersection, or portal.
- All topology-changing operations are undoable as a single transaction.
- Generated geometry should never obscure the source centerline/control structure when edit mode is active.

3.3 Canvas navigation

- Infinite pan/zoom canvas.
- Mouse-centered zoom and keyboard framing commands.
- Saved view bookmarks.
- Optional mini-overview navigator.
- Scale bar and cursor coordinates.
- LOD override control: Auto / Network / Envelope / Lane / Micro.

4. Coordinate, Units & Precision Model

The canonical geometry space uses double-precision world coordinates expressed internally in meters unless a future implementation has a compelling technical reason to choose another base unit. Display units are independent.

Capability	Specification
World origin	Project-configurable logical origin; maps may have local transforms relative to parents.
Units	Metric/imperial display; calculations use canonical base units.
Angles	Degrees in UI; radians permitted internally.
Spline parameterization	Operations exposed by physical arc length where possible, not unstable raw control-point indices.
Tolerance	Project-level geometry tolerance for snapping, equality, and topology tests.
Grid	Adaptive major/minor spacing; snap independently toggleable.

5. Vector Geometry & Drawing Tools

5.1 Core tools

Tool	Behavior
Bezier Pen	Place anchors; drag tangents; sharp/smooth/symmetric/asymmetric nodes.
Interpolating Spline	Place points through which a smooth spline passes.
Polyline	Connected straight segments.
Fixed-Radius Path	Polyline-like drawing with real-world fillet radius constraints.
Freehand-to-Spline	Sample gesture, simplify, fit editable spline.
Polygon/Rectangle/Circle	General map geometry.
Offset	Generate exact parallel path/outline.
Split/Trim/Join	Edit generic vector paths; RoadSegment split is a separate semantic operation.

Simplify	Reduce anchors while preserving a configurable geometric error bound.
----------	---

5.2 Snapping

- Grid and guides
- Object vertices and anchors
- RoadSpline start/end sockets
- Arbitrary road attachment sockets
- Lane boundaries and lane centerlines
- Intersection/junction anchors
- Building edges and parcel boundaries
- Portal/submap anchors
- Angular constraints and perpendicular/parallel inference

6. RoadSpline System

CORE RoadSpline is the authoritative continuous, directional geometric path of a road corridor. It is not the rendered road polygon and it is not synonymous with a RoadSegment.

6.1 Required RoadSpline data

Field	Purpose
id	Stable unique identity.
centerline	Bezier/spline control geometry.
direction	Canonical Start → End orientation.
segments	Ordered, non-overlapping RoadSegments covering the spline domain.
elevationProfile	Optional Z/grade definition independent of logical SpatialLevel.
roadClass/style	Optional defaults inherited by segments.
networkRefs	Connections/junctions attached to the spline.
metadata	Extensible project-defined values.

6.2 Invariants

- Segments cover the RoadSpline continuously unless an explicit future feature allows gaps.

- Segment order follows canonical spline direction.
- Segment boundaries are represented by stable arc-length positions or robust normalized positions with migration rules.
- Editing centerline geometry must preserve segment boundaries by physical position or an explicit reparameterization policy.
- Reversing a RoadSpline is an explicit operation because left/right semantics and directional lane interpretation are affected.

6.3 Editor appearance

In Road edit mode the centerline is visually prominent, with anchors, tangents, direction arrows, segment-boundary handles, and start/end sockets. Generated road envelopes and lanes remain visible but subordinate. Segment labels may show index/name and length at suitable zoom.

7. RoadSegment System

CORE When a RoadSpline is first created, one RoadSegment spans 100% of its centerline. Splitting creates two independently configurable intervals while preserving the parent RoadSpline.

7.1 Split operation

1. User invokes Split Segment at a cursor/arc-length position.
2. System validates the split is not within endpoint tolerance.
3. Original segment is replaced by two segments sharing the split boundary.
4. Both children initially inherit the original segment configuration and metadata according to schema-defined copy rules.
5. Lane identities are preserved or forked using stable lineage IDs so connectivity can be repaired deterministically.
6. Dependent derived geometry and network relationships are invalidated and recomputed.
7. The operation is committed as one undo transaction.

7.2 Merge operation

Adjacent segments may merge when their cross-section/network states are compatible or after the user chooses a conflict-resolution policy. A merge must never silently discard non-equivalent lane, metadata, or connectivity information.

7.3 Segment boundary editing

A segment boundary may slide along the RoadSpline. The inspector should expose exact station/arc-length values. Moving a boundary changes the intervals, not the parent spline geometry.

7.4 Segment-local properties

- Cross-section elements and lanes
- Road classification/style overrides

- Surface/markings defaults
- Speed/design metadata if defined by project
- Spatial/elevation overrides where appropriate
- Procedural roadside rules
- Custom project metadata

8. Lane & Cross-Section Architecture

8.1 Cross-section model

Each RoadSegment owns ordered cross-section elements positioned relative to the parent RoadSpline orientation. The system should support Left, Center, and Right groups so medians or center elements are not forced into an arbitrary lane side.

```

LEFT OF CENTERLINE      CENTER      RIGHT OF CENTERLINE
[Sidewalk][Buffer][Lane][Lane] [Median] [Lane][Lane][Bike][Sidewalk]

```

8.2 Element schema

Property	Notes
type	TravelLane, TurnLane, Parking, Bike, Bus, Shoulder, Sidewalk, Median, Buffer, Verge, Rail, Custom.
width	Real-world width; may be constant or profile-driven.
side/order	Deterministic ordering relative to centerline.
direction	Forward, Reverse, Bidirectional, None, or project-defined.
markings/style	Presentation and export metadata.
start/end behavior	Full-length, taper, begin/end station, transition ownership.
connectivity role	Whether element participates in lane network.
metadata	Extensible project schema.

8.3 Width profiles

Constant width is the default. Variable width uses a profile over segment arc length. Control points should be sparse semantic parameters, not necessarily identical to rendering samples. Interpolation mode may be linear, smooth, or constrained by a transition object.

8.4 Cross-Section Editor UX

- Graphical strip showing elements in physical order and proportional width.
- Add, delete, duplicate, reorder, mirror, and convert element type.
- Numeric width fields and optional drag handles.
- Left/Full/Right focus modes.

- Templates for common sections.
- Warnings for zero/negative width, impossible taper, or connectivity mismatch.
- Selecting an element highlights its generated footprint on the map.

9. Derived Road Geometry

9.1 Source vs derived data

Authoritative source	Derived output
RoadSpline centerline	Sampled frames/transforms
RoadSegment intervals	Segment clipping ranges
Cross-section definitions	Lane centerlines/boundaries and road envelope
Width profiles	Interpolated offsets
Network transitions	Merge/diverge transition curves
Junction definitions	Intersection polygons/connection curves
Styles	LOD-specific strokes, fills, markings

9.2 Generation rules

- Offsets should be calculated in a stable local frame along the centerline.
- Sharp curvature must detect self-intersection or offset inversion and warn rather than generate silently corrupt geometry.
- Rendering tessellation density is view/performance dependent and must not change semantic geometry.
- Derived geometry should be cacheable and invalidated at the smallest practical dependency scope.

10. Semantic Zoom & Rendering

10.1 LOD stages

LOD	Primary meaning	Typical display
Network	Connectivity/topology	Single simplified road strokes, major labels.
Envelope	Physical footprint	Road width, medians, major sidewalks.
Lane	Operational structure	Individual lanes, transitions, lane directions.

Micro	Street detail	Markings, arrows, parking stalls, crosswalks, curbs, repeated props.
-------	---------------	--

10.2 Rules

- LOD changes representation only; it does not mutate source geometry.
- Thresholds are style/project configurable.
- Transitions should fade or cross-dissolve when feasible to reduce visual popping.
- Manual LOD override is required for debugging and authoring.
- Labels use independent collision/priority logic rather than naive scale transforms.

11. Transport Network & Connectivity

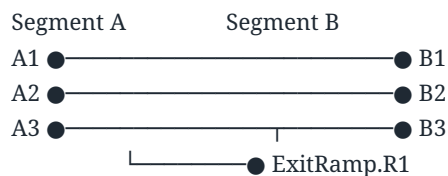
CORE Geometry answers “where is it?” Connectivity answers “what leads to what?” Two roads can overlap geometrically without connecting, and lanes can connect semantically even when their generated transition geometry is complex.

11.1 Connection nodes and sockets

- Every RoadSpline start/end exposes a socket.
- Optional sockets may exist at arbitrary stations along a RoadSpline or at segment boundaries.
- Sockets expose orientation, spatial level/elevation, cross-section signature, and compatible connection roles.
- Dragging compatible sockets together previews Connect, Connect + Transition, Create Junction, or Keep Separate.

11.2 LaneConnection

LaneConnection is a directed semantic edge from one lane endpoint/port to another. It may represent continuation, turn movement, merge participation, diverge participation, or portal linkage. Stable lane IDs are therefore essential.



11.3 Automatic mapping

When adjacent cross-sections are compatible, the editor proposes deterministic lane mappings using side, order, direction, and type. The proposal must be inspectable and overridable. Ambiguous mappings require user confirmation rather than silent guessing.

12. RoadTransition System

12.1 Purpose

RoadTransition is a first-class semantic object that describes how one or more input cross-section elements become one or more outputs over a defined physical length. It owns the relationship; generated taper/merge geometry is derived.

12.2 Built-in transition families

Transition	Semantic behavior
Continuation	1:1 lane continuity.
Lane Add	One or more lanes appear over a taper/transition length.
Lane Drop	Lane terminates or merges into neighbor.
Merge	Separate road/lane stream joins host stream.
Diverge	Host stream branches into ramp/secondary road.
Cross-section Morph	Widths/types/order change between segment configurations.
On-ramp	Ramp + acceleration/merge lane pattern.
Off-ramp	Diverge + optional deceleration lane pattern.
Custom	Explicit input/output lane mapping and geometric rules.

12.3 Transition parameters

- Start/end stations
- Transition length
- Input/output lane ports
- Taper mode
- Minimum radius/curvature constraints
- Width interpolation
- Host/branch side
- Style/markings behavior
- Validation constraints

13. Junction & Intersection System

13.1 Junction definition

A Junction is a network object that coordinates multiple road approaches and their lane movements. It is distinct from the generated intersection polygon.

13.2 Supported forms

- Simple endpoint join
- T-junction
- Four-way/multi-leg junction
- Roundabout
- Ramp terminal
- Highway merge/diverge node
- User-defined arbitrary junction

13.3 Junction geometry

Approach road envelopes are clipped to junction boundaries; connection curves and fill regions are generated from semantic lane movements and corner/radius rules. The user may override curb/corner geometry without destroying lane connectivity.

13.4 Lane movement matrix

Advanced junction editing exposes a matrix/graph of permitted lane movements. This same data can later support traffic AI, route validation, or engine export without making those systems core editor responsibilities.

14. Connectivity Graph Editor

The application remains map-first. A node graph is an advanced secondary view for inspecting complex connectivity, not the default way to draw roads.

- Selecting a physical object highlights its graph node and vice versa.
- Graph nodes represent roads/segments, junctions, transitions, merges, diverges, portals, and prefab ports.
- Lane-level expansion is available on demand to avoid graph explosion.
- Broken references and ambiguous mappings are visibly flagged.
- Graph layout is presentation-only and does not affect world geometry.

15. Parametric Prefab System

15.1 Definition

A Prefab is a reusable, parameterized assembly of source objects and relationships. It may contain RoadSplines, RoadSegments, lanes, transitions, junctions, spatial-level rules, metadata, procedural objects, and attachment ports.

15.2 Scale of prefabs

- Micro: 3→2 lane transition, turn pocket, bus bay.
- Meso: roundabout, cul-de-sac, highway exit, on-ramp.
- Macro: diamond, folded diamond, trumpet, cloverleaf, partial cloverleaf, SPUI, diverging diamond, custom interchange.

15.3 Ports

Prefab ports expose semantic attachment contracts such as Highway West, Highway East, Arterial North, Arterial South. Each port may specify expected lane signatures, direction, spatial level, elevation/clearance constraints, and whether adaptation is permitted.

15.4 Drag-along-host workflow

8. User drags an interchange/ramp prefab over a candidate host RoadSpline.
9. Prefab snaps to the host by station and orientation and previews its footprint.
10. System searches for compatible crossing/target roads within a configurable radius.
11. Elevation and spatial-level relationships determine whether the crossing is a junction, overpass, or underpass.
12. Ports negotiate lane/cross-section compatibility and preview required adapters/transitions.
13. User adjusts parameters such as ramp lane count, taper length, radius, and clearance.
14. Commit instantiates a live PrefabInstance retaining its parameter linkage.

15.5 Editing and detachment

Prefab instances remain parametrically editable. “Make Unique/Detach” converts an instance into ordinary source objects while preserving geometry and connectivity. Destructive detachment must be explicit.

16. Elevation & Spatial Levels

16.1 Two elevation concepts

Concept	Meaning
Numeric elevation / Z	Physical height and grade in world units.
SpatialLevel	Logical navigational/display stratum such as Ground, +1 Overpass, -1 Tunnel.

RULE SpatialLevel must never substitute for actual elevation. Two objects can share a SpatialLevel yet have different Z, and an overpass relationship can be inferred from both logical and physical information.

16.2 Capabilities

- Elevation profiles along RoadSplines
- Grade display and warnings
- Bridge/tunnel flags or specialized segment roles
- Ramps connecting levels
- Overpass/underpass non-connection rules
- Vertical clearance metadata

- Active-level isolation
- Ghosting adjacent levels
- Player-context visibility preview

17. Maps, Submaps & Portals

17.1 Submap purpose

Submaps prevent dense interiors or local facilities from polluting the global map while preserving semantic linkage. A child map may have its own coordinate transform, levels, layers, styles, and LOD thresholds.

17.2 Examples

- World → Downtown → Mall → Parking Garage → B2
- World → Airport → Terminal → Concourse
- World → Transit Station → Underground Platform
- World → Building → Interior floor

17.3 Portals

Portal objects link navigable locations across maps or spatial levels. A portal stores source/target anchors, directionality, transform relationship, display rules, and optional transition metadata. Road ramps can be continuous geometry; elevators/doors may be portal-only connections.

18. Buildings, Blocks & Parcels

18.1 Building objects

- Editable footprint polygon
- Height/floor metadata
- Type/classification and custom metadata
- Entrances/portal anchors
- Style and label rules
- Optional child interior map

18.2 Road-adjacent building tool

The user selects a road side or frontage interval and specifies setback, frontage width/range, depth, gap, and corner behavior. Buildings are generated as editable parametric footprints aligned to the road envelope, not the centerline.

18.3 Blocks and parcels

- Detect closed regions from road envelopes/network
- Generate block polygon
- Inset by sidewalk/setback rules
- Subdivide into parcels
- Generate plausible footprints

- Preserve generated parameters until detached

19. Environmental & General Geometry

System	Capabilities
Water	Polygons, coastlines, lakes; style/metadata.
River/Canal	Spline with variable width and optional banks.
Rail	Spline/corridor model; may later adopt lane-like tracks.
Pedestrian paths	Lightweight spline corridors.
Terrain regions	Polygon classifications, contours, elevation references.
Barriers/walls	Spline-based linear features.
Parks/green space	Polygonal region with style/metadata.

20. Procedural Spline-Distributed Objects

Inspired by the thesis FSplineActor concept, any compatible spline/corridor can own repeat rules for symbolic or exportable objects.

Parameter	Example
Asset/type	Streetlight, tree, guardrail post, bollard, sign.
Spacing	30 m.
Start/end offset	Skip first/last 10 m.
Lateral offset	2 m from road edge.
Orientation	Follow spline tangent, face road, world-up.
Side	Left/right/both.
Exclusions	Suppress near junctions, driveways, portals.
Randomization	Optional bounded variation for visual planning.

21. Game-Agnostic Semantic Objects

21.1 Primitive semantic types

Type	Geometry
POI	Point/anchor
Zone	Polygon/multipolygon
Route	Ordered path/network reference
Anchor	Point with orientation/role
Custom Object	Project-defined geometry/schema where supported

21.2 Project schemas

Projects define categories, fields, validation, icons, colors, and defaults. Example: a racing game can define RaceStart, Garage, PoliceSpawn, and FactionZone; another project can define QuestGiver, EnemySpawn, SafeZone, and FastTravel without changing core code.

21.3 Property field types

- String / multiline text
- Integer / float / distance / angle
- Boolean
- Enum / multi-enum
- Color / icon
- Object reference
- Map/submap reference
- Asset/external ID
- Tag set
- Optional expression/derived field in a future extension

22. Display Layers, Groups, Tags & Search

22.1 Display layers

Display layers control visibility, lock state, style overrides, opacity, export inclusion, and authoring organization. They do not determine elevation or map hierarchy.

22.2 Search and queries

- Text/name/tag search
- Type and layer filters
- Property comparisons such as $\text{laneCount} \geq 6$
- Spatial filters: in viewport, in polygon, within distance

- Network filters: disconnected, dead-end, connected to selected
- Saveable smart selections in a future version

23. Reference & Drafting Workflow

23.1 Reference overlays

- Raster image import
- Opacity/visibility/lock
- Scale calibration from known distance
- Rotation and translation
- Crop/mask
- Multiple reference layers
- Optional georeference metadata without requiring GIS

23.2 Drafting aids

- Guides and rulers
- Angle/length constraints
- Parallel/perpendicular inference
- Exact radius entry
- Measurement annotations
- Notes and callouts
- View bookmarks

24. Measurement, Analysis & Validation

24.1 Measurements

- Straight/path distance
- Area/perimeter
- RoadSpline and RoadSegment length
- Lane-kilometers
- Curve radius/curvature
- Elevation/grade
- Building/parcel area
- Road and intersection density

24.2 Validation categories

Category	Examples
Geometry	Self-intersecting offset, negative width, impossible radius.
Segmentation	Overlapping/gapped segment intervals.
Connectivity	Orphan lane, incompatible direction, broken node

	reference.
Transition	Insufficient length, unmapped lane, invalid merge/diverge.
Spatial	Unexpected same-level crossing, insufficient clearance metadata.
Submap	Broken portal target, circular hierarchy.
Schema	Missing required custom property, invalid enum/reference.

24.3 Diagnostics UX

Warnings should be non-modal whenever possible, appear both in a diagnostics panel and on-canvas, and include a “Select/Fix” action. Fatal integrity errors block export/save only when data loss or corruption would result; design warnings should remain advisory.

25. Styling & Cartographic Presentation

25.1 Style system

- Project theme and named style presets
- Per-object-type default styles
- Layer overrides
- Selection/hover states independent of export style
- LOD-specific representation
- Label fonts, priorities, halos, and collision rules
- Road names following splines
- Custom icons for semantic objects

25.2 Presets

- Technical/CAD
- Blueprint
- Minimal light
- Minimal dark
- Game minimap
- Presentation map
- Reference-overlay

26. Project File & Data Model

26.1 Serialization goals

- Human-debuggable where practical
- Stable IDs

- Versioned schema
- Forward migration support
- Separation of source data from disposable caches
- Deterministic ordering where it improves diffs/version control
- External binary assets referenced rather than embedded when appropriate

26.2 Conceptual schema

```
Project {
  schemaVersion, units, definitions, styles, maps[], libraries[]
}
Map { id, name, transform, spatialLevels[], displayLayers[], objects[], childMaps[] }
RoadSpline { id, centerline, direction, segmentIds[], elevationProfile, metadata }
RoadSegment { id, roadSplineId, startStation, endStation, crossSection, metadata }
CrossSectionElement { id, typeId, side, order, widthProfile, direction, style, metadata }
LaneConnection { id, fromLanePort, toLanePort, movementType, metadata }
RoadTransition { id, inputs[], outputs[], stations, parameters }
PrefabInstance { id, prefabDefinitionId, hostBindings, parameters, overrides }
```

26.3 Stable identity

IDs must survive ordinary geometric edits. Array index is never identity. Splitting and merging require explicit lineage/migration behavior so references can be updated or flagged instead of silently pointing at the wrong lane.

27. Dependency & Rebuild Architecture

27.1 Dependency flow

```
Edit centerline
→ RoadSpline arc-length/frame cache
→ affected RoadSegments
→ cross-section/lane derived geometry
→ attached transitions/junctions
→ network validation
→ road-adjacent/procedural dependents
→ LOD render caches
```

27.2 Dirty-region strategy

The implementation should mark the smallest affected object/range dirty rather than rebuilding the entire world. A control-point edit on one RoadSpline should not regenerate unrelated roads or submaps.

27.3 Background work

Expensive tessellation, block generation, validation, or import/export work should be cancellable and may run in background workers. Authoritative topology mutations must remain serialized/transactional to avoid race conditions.

28. Undo, Transactions & Non-Destructive Editing

- Every user-visible command produces a reversible transaction.
- Compound operations such as placing an interchange prefab undo in one step.
- Derived cache rebuilds are not separate undo events.
- Property drags may coalesce into one transaction.
- Prefab detachment, segment merge, and schema deletion require explicit confirmation if references will be affected.
- Autosave stores authoritative state, not transient render caches.

29. Performance & Large-World Strategy

29.1 Principles

- Viewport culling and spatial index for map objects.
- LOD-based rendering independent of semantic precision.
- Lazy loading for child maps/submaps.
- Chunked/cached road geometry.
- Incremental network validation.
- Avoid per-frame recomputation of unchanged geometry.
- Use simplified hit-testing structures at distant zoom levels.

29.2 Suggested indexes

- R-tree/quadtrees-like spatial index for 2D objects
- Graph adjacency index for network traversal
- ID lookup table
- RoadSpline station/segment interval index
- Optional tile cache for raster/reference layers

30. Import, Export & Extensibility

30.1 Core exports

Format	Purpose
PNG	Raster presentation.
SVG	Vector map presentation.
PDF	Printable/shareable maps.
JSON	Full or selected structured project data.
CSV	Tabular metadata/statistics.
GeoJSON-style	Interchange with spatial tooling where semantics permit.

30.2 Exporter/plugin architecture

Game-engine integration should be implemented as adapters/exporters. An Unreal exporter can translate RoadSplines, lanes, metadata, and prefabs into engine-specific assets without making Unreal concepts part of the core model. Unity or custom exporters can coexist.

30.3 Future imports

- SVG paths/polygons
- GeoJSON
- OpenStreetMap-derived data
- CSV point datasets
- GIS formats through plugins
- Existing project-specific map data

31. Detailed Editor Workflows

31.1 Create and split a road

15. Activate Road tool and choose Bezier, fixed-radius, polyline, or spline mode.
16. Draw centerline. System creates RoadSpline + one full-length RoadSegment using active road/cross-section preset.
17. Select segment and invoke Split Segment at cursor or exact station.
18. Two child segments inherit configuration.
19. Edit Segment B cross-section independently; generated road updates immediately.

31.2 Add a turn lane

20. Split approach region if a distinct configuration interval is required.
21. Open Cross-Section Editor for approach segment.
22. Add Turn Lane element on desired side/order.
23. Create or choose a Lane Add transition from upstream segment.
24. Map turn lane through the Junction movement editor.
25. Validation confirms lane start, taper length, and downstream movement.

31.3 Connect dead-end roads

26. Drag Road B endpoint near Road A endpoint.
27. Socket snap preview compares elevation, direction, and cross-section signatures.
28. If compatible, choose Connect.
29. If lane counts/types differ, choose Connect + Transition and resolve mapping.
30. System creates network connection and derived geometry.

31.4 Create an off-ramp

31. Choose Off-Ramp transition/prefab and drag along highway RoadSpline.
32. Snap to host station and side.

33. Set deceleration lane/taper length, ramp lane count, width, and radius.
34. System splits host segments as necessary, creates diverge semantics, and generates ramp RoadSpline.
35. Connect ramp endpoint to downstream road/junction or leave as an explicit dead end.

31.5 Place a diamond interchange

36. Draw highway and arterial with correct elevations/spatial levels.
37. Drag Diamond Interchange prefab along highway.
38. Prefab detects crossing arterial and previews four ports/ramps.
39. Resolve lane signatures and vertical clearance.
40. Adjust ramp radius, acceleration/deceleration lengths, and terminal junction style.
41. Commit as live PrefabInstance.
42. Open Connectivity Graph for advanced lane-movement inspection if needed.

31.6 Create parking-garage submap

43. Create building/garage footprint and entrance anchor on parent map.
44. Create child map “Parking Garage”.
45. Add floors as SpatialLevels or nested maps according to desired workflow.
46. Draw lane-scale internal roads/ramps/parking geometry.
47. Create portals or continuous ramp connections between parent and child contexts.
48. Use Player-Context Preview to verify map switching/visibility.

32. Default Tool & Command Inventory

Category	Commands
Selection	Select, lasso, box select, isolate, hide, lock, frame.
Road	Draw, reverse, add point, remove point, split segment, merge segments, cross-section editor.
Network	Connect, disconnect, create socket, junction, transition, lane mapping, graph view.
Prefab	Place, edit parameters, expose ports, save selection as prefab, detach.
Vector	Bezier, spline, polyline, fixed radius, freehand, offset, trim, join, simplify.
Urban	Building, road-adjacent building, block, parcel, entrance.
Semantic	POI, zone, route, anchor, custom object.
Spatial	Add level, set elevation, bridge, tunnel, portal, submap.
Drafting	Measure, guide, annotation, reference image, calibration.

33. MVP, V1 & Future Scope

33.1 MVP

- Project/map save-load with stable IDs and schema versioning.
- Infinite 2D canvas, real-world coordinates, grid/snap, measurement.
- Bezier/polyline/fixed-radius RoadSpline drawing.
- RoadSpline → RoadSegment model with split/merge.
- Lane/cross-section editor with real widths and basic lane types.
- Derived road envelope/lane rendering with semantic zoom.
- Basic endpoint/T-junction connectivity and lane continuation.
- Display layers and basic object hierarchy.
- Buildings/polygons, generic POIs/zones, reference images.
- Undo/redo, autosave/recovery, PNG/SVG/JSON export.

33.2 V1

- Full transition system: lane add/drop, merge, diverge, on/off ramps.
- Advanced junction editor and lane movement graph.
- Spatial levels, bridges/tunnels, elevation profiles.
- Submaps and portals.
- Parametric prefab library and user-created prefabs.
- Road-adjacent buildings, blocks/parcels.
- Procedural spline-distributed objects.
- Advanced validation/statistics and style system.
- Plugin/exporter API.

33.3 Future

- Advanced GIS/OSM import
- Traffic simulation preview
- 3D terrain/road preview
- Collaborative editing
- Scripting/automation API
- Constraint-solving interchange design
- Procedural city/block generation
- Live engine synchronization

34. Explicit Non-Goals & Guardrails

- Do not turn the primary editor into a photorealistic 3D scene editor.
- Do not make satellite imagery the default canvas; it is a reference layer.

- Do not store road appearance as manually painted polygons when it can be derived from semantic source data.
- Do not hard-code game-specific POI/event/faction concepts into the core application.
- Do not conflate a RoadSpline control-point span with a RoadSegment; segments are semantic intervals and may not correspond 1:1 to spline control points.
- Do not infer a network connection merely because two roads cross in XY.
- Do not use lane array indices as persistent identities.
- Do not allow prefab convenience to hide invalid topology; generated results must remain inspectable and validatable.

35. Technical Risks & Design Questions

Risk / question	Mitigation / decision direction
Offset curves self-intersect on tight radii	Detect curvature/width violations; warn and offer local geometry resolution.
Segment boundaries drift after spline edits	Store robust station anchors/normalized positions with explicit reparameterization behavior.
Lane identity after split/merge	Stable IDs + lineage mapping; never rely on index.
Prefab adaptation becomes too magical	Use explicit port contracts, preview, and user-confirmed adapters.
Graph complexity explodes	Collapsed road-level graph by default; expand to lanes only on demand.
Vertical model ambiguity	Keep numeric Z and logical SpatialLevel separate.
Submap coordinate complexity	Parent transform + explicit portals; avoid hidden coordinate conversion.
Large-world performance	Spatial indexing, dirty-range rebuilds, LOD rendering, lazy submaps.
Game-specific pressure on core model	Custom schemas/export plugins rather than core special cases.

36. Glossary

Term	Canonical meaning
RoadSpline	Continuous directional centerline defining a road corridor.
RoadSegment	Independently configurable interval along one RoadSpline.
Station	Physical distance along a RoadSpline from its canonical

	start.
Cross-section element	Ordered component of a RoadSegment width: lane, median, sidewalk, etc.
Lane	Cross-section element participating in transportation connectivity.
Lane port	Semantic endpoint/attachment point of a lane for network connections.
Connection node	Network entity relating road/lane endpoints or attachment points.
RoadTransition	Semantic mapping between differing input/output lane configurations over distance.
Junction	Network object coordinating multiple road approaches and movements.
Prefab	Reusable parameterized assembly of source objects and relationships.
SpatialLevel	Logical vertical/navigational stratum; not the same as numeric elevation.
DisplayLayer	Visibility/organization layer; not the same as SpatialLevel.
Submap	Nested map with its own detailed geometry and context.
Portal	Semantic connection between map/submap or level contexts.
Semantic zoom	LOD system that changes representation according to meaning and scale.
Derived geometry	Regenerable visual/physical geometry calculated from authoritative semantic source data.

Appendix A. Thesis-to-New-Architecture Mapping

2023 thesis concept	New editor evolution
SBRoadSpline directional Bezier RoadSpline	RoadSpline remains directional centerline; standalone, engine-agnostic.
LeftLanes / RightLanes arrays	RoadSegment-owned ordered Left/Center/Right cross-section elements.
FLaneSpline LaneWidth + ControlPoints	Stable lane element + width profile over segment station.
SplineTransforms sampling	Derived spline frame/cache; tessellation independent of

	semantics.
SBIntersection + FRoadConnection	General Junction + lane movement/connectivity model.
FSideConnection	Arbitrary sockets, T-junctions, merges/diverges, RoadTransitions.
FSplineActor repeated props	Generic spline-distributed object rules.
Geometry Script mesh output	Editor produces 2D vector/semantic geometry; engine-specific geometry delegated to exporters/plugins.

Appendix B. AI Handoff Contract

USE THIS SECTION WHEN STARTING ANOTHER AI CHAT Provide this document as the primary design source. Ask the AI to preserve the canonical terminology and invariants below unless you explicitly request a redesign.

- Treat RoadSpline, RoadSegment, Lane/CrossSectionElement, RoadTransition, Junction, SpatialLevel, DisplayLayer, Submap, Portal, and Prefab as distinct concepts.
- Assume one RoadSegment spans a new RoadSpline until explicitly split.
- Assume lane/cross-section geometry is segment-owned and road polygons are derived.
- Assume semantic lane connectivity is separate from visual overlap.
- Assume numeric elevation and logical SpatialLevel coexist.
- Assume the editor is 2D-first and game-agnostic.
- Prefer reusable parametric systems over special-case features.
- When proposing code, preserve stable IDs, undoable transactions, dependency invalidation, and schema migration.
- When proposing UI, keep the map canvas primary; node graphs and 3D previews are secondary specialist views.
- Flag any proposed change that violates an invariant before implementing it.

Suggested context prompt

Use the attached Parametric World Cartography Editor Master Design Specification as the canonical project architecture. Preserve its terminology and invariants. Before proposing changes, identify which objects and dependencies are affected. Distinguish authoritative source data from derived geometry. Do not silently introduce game-specific concepts into the core model.

Appendix C. Source Basis

This master specification synthesizes the design conversation leading to September 2026, the supplied Speedbreak comprehensive project summary for the original motivating use case, the supplied 2023 senior thesis “Developing a Comprehensive Tool for User-Oriented Vehicle and Road Simulation,” and the generated GUI concept used to explore interface direction. Speedbreak-specific

gameplay semantics are intentionally excluded from the core editor architecture so the application remains game-agnostic.

The thesis material particularly informs the directional RoadSpline, lane-offset generation, variable lane width/control concepts, endpoint intersections, side connections, and repeated spline objects. The present specification intentionally changes the ownership model by introducing explicit RoadSegments and a semantic network layer.